

MO INTEGRATIONS · TECHNICAL TRAINING

Flutter Mobile Application Development · 5-Day Programme

Day 1

Foundations & Dart

Environment · Dart primer · Async Dart · First Flutter app

Anchor project: white-label motor-insurance quote app · Instructor: Clynton Botterill

Day 1 agenda

1

09:00 - 10:30

Setup & tooling

- What Flutter is and how it renders
- SDK, IDE, emulator, devices
- Project anatomy, hot reload
- Lab 1.1 - toolchain verified

2

10:45 - 12:30

Dart primer

- Mapped from Java / C# / TypeScript
- Null safety, named params, classes
- Records, sealed classes, patterns
- Lab 1.2 - quote domain model

3

13:15 - 14:30

Async Dart

- Future vs Promise / Task
- async / await, error handling
- Streams vs Observables
- Lab 1.3 - async quote service

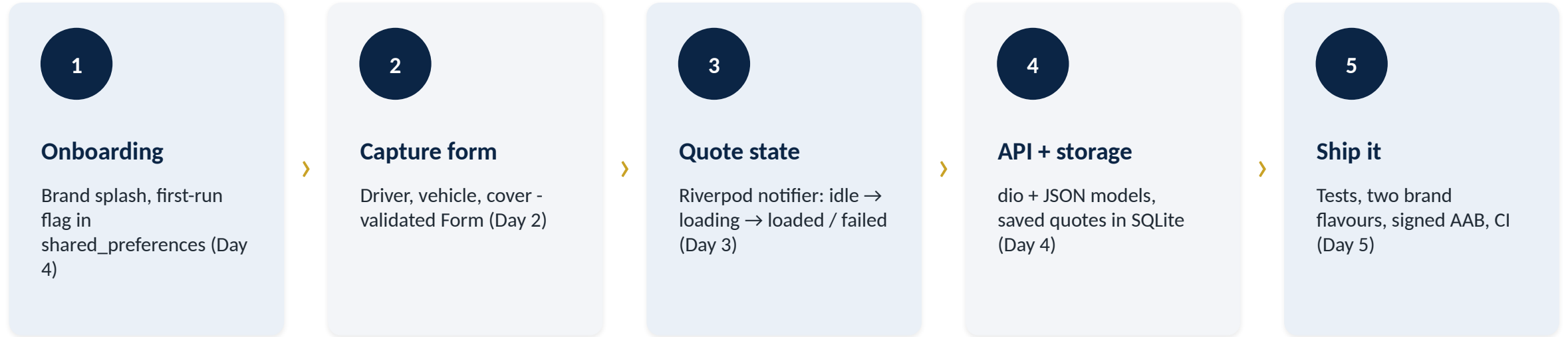
4

14:45 - 16:00

First Flutter app

- Theory to 15:15: widget tree, three trees, main.dart
- Stateless vs Stateful, lifecycle, DevTools
- Lab 1.4 - 15:15 to 15:55
- Recap 15:55 - 16:00

The app we build this week



One codebase, many brands. Every lab adds a piece; Friday afternoon is a project clinic against your real insurance app.

Where the survey said we should spend the time

1.6

Publishing to stores
(lowest) → Day 5

1.8

State management
→ Day 3

2.2

Widgets & layout
→ Day 2

4.0+

OOP, REST/JSON, Git
→ taught as Flutter idiom only

Averages from the pre-course skills survey (1 = never done it, 5 = very confident). We move fast where you are strong and slow where you are not. Dart ranged from 1 to 5, so this morning's primer is brisk with a stretch lane.

MODULE 1 • 09:00 - 10:30

Setup & tooling

By the end of this module you can

- Explain how Flutter renders and why it has no bridge
- Install and verify the SDK on macOS or Windows
- Run an app on an emulator and a physical device
- Navigate a Flutter project and use hot reload

What Flutter is

A UI toolkit from Google. One Dart codebase → Android, iOS, web, Windows, macOS, Linux.

It owns every pixel. Flutter does not use native buttons or text fields. It draws its own on a GPU canvas via Impeller.

Result: identical UI on every platform, no bridge, no platform-specific UI bugs.

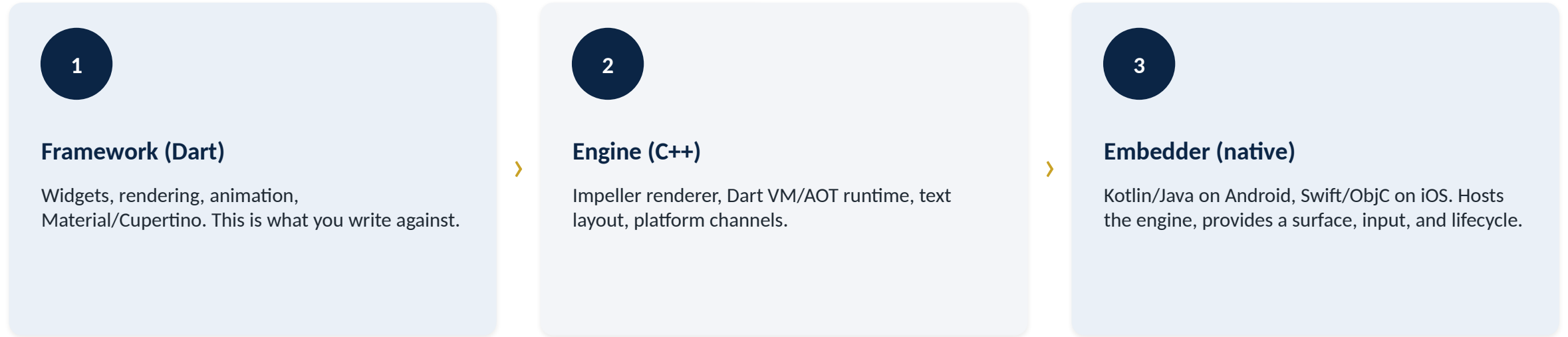
Trade-off: the app does not automatically look "native"; you choose Material or Cupertino.

Who uses it: Google Pay, BMW, Toyota, Alibaba, Nubank, eBay Motors.

```
// Everything is a widget - including layout
Scaffold(
  appBar: AppBar(title: Text('Get a quote')),
  body: Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        Text('Monthly premium'),
        Text('R 1 450', style: TextStyle(fontSize: 40)),
        FilledButton(
          onPressed: () {},
          child: Text('Accept'),
        ),
      ],
    ),
  ),
)
```

Dart

Architecture: three layers



You spend 99% of your time in the top layer. Platform channels (Day 5) let Dart call into the embedder when you need a native API the plugin ecosystem does not cover.

Flutter vs React Native vs native

	Flutter	React Native	Native (Kotlin / Swift)
Language	Dart	JavaScript / TypeScript	Kotlin, Swift
UI rendering	Own canvas (Impeller) - pixel-identical	Native views via JSI / Fabric	Native views
Performance	AOT compiled, no bridge	JS engine + native, near-native	Best
Look & feel	Same everywhere; Material or Cupertino	Platform-native by default	Platform-native
Web / desktop	Web, Windows, macOS, Linux	Web via react-native-web; desktop limited	No
OTA updates	Shorebird (3rd party)	Expo EAS Update built in	No
Ecosystem	pub.dev, ~50k packages	npm, huge	Platform SDKs
Team fit here	C# / Java / TS devs - Dart is a small jump	JS/TS devs	Two codebases

Dart: JIT in development, AOT in production

Development: Dart runs on a JIT VM. Change code → hot reload injects it into the running app in under a second, keeping state.

Release: ahead-of-time compiled to native ARM / x64. No interpreter, no bridge, predictable performance.

Sound null safety: nullability is part of the type. The compiler, not you, proves a value cannot be null.

Familiar: C-style syntax, classes, generics, async/await. If you read TypeScript or C#, you can read Dart today.

```
// Same file, two lives
// dev:      flutter run           → JIT, hot reload
// release: flutter build apk      → AOT native code
```

```
void main() => runApp(const QuoteApp());
```

```
// Null safety is in the type system:
String name = 'Sive';    // never null
String? nickname;        // may be null
print(nickname?.length ?? 0);
```

Dart

Hot reload keeps app state. Hot restart (capital R) resets it. Full rebuild is only needed for native code or new plugins.

Install the Flutter SDK

Two rules: no spaces in the path, not inside Program Files. Then add bin to PATH and open a new terminal.

macOS • zsh

```
# 1. Download stable zip (Apple Silicon or Intel)
# docs.flutter.dev/get-started/install/macos
mkdir -p ~/development
cd ~/development
unzip ~/Downloads/flutter_macos_*-stable.zip

# 2. PATH - add to ~/.zshrc
echo 'export PATH="$HOME/development/flutter/bin:$PATH"' >>
~/.zshrc
source ~/.zshrc

# 3. Verify
flutter --version
# Xcode + CocoaPods needed only for iOS builds:
xcode-select --install
sudo gem install cocoapods
```

Windows • PowerShell

```
# 1. Download stable zip
# docs.flutter.dev/get-started/install/windows
# Extract to C:\src\flutter (NOT Program Files)
Expand-Archive $HOME\Downloads\flutter_windows_*-stable.zip
C:\src

# 2. PATH - user environment variable
# Settings > System > About > Advanced system settings
# > Environment Variables > User > Path > New:
# C:\src\flutter\bin
# or from PowerShell (user scope):
$u = [Environment]::GetEnvironmentVariable("Path", "User")
[Environment]::SetEnvironmentVariable("Path",
"$u;C:\src\flutter\bin", "User")

# 3. Open a NEW PowerShell, then:
flutter --version
```

Windows: enable long paths once - git config --system core.longpaths true. Also exclude the project folder and C:\src\flutter from antivirus scanning or Gradle builds crawl.

Android toolchain: Android Studio + SDK + emulator driver

Android Studio is the SDK manager and the emulator host, even if you code in VS Code.

macOS • zsh

```
# Install Android Studio (developer.android.com/studio)
# First-run wizard: Standard install → accepts SDK,
#   SDK Platform, Android Virtual Device.

# Android Studio → More Actions → SDK Manager
#   SDK Platforms: tick the latest stable API
#   SDK Tools:   Android SDK Build-Tools
#               Android SDK Command-line Tools (latest)
#               Android Emulator
#               Android SDK Platform-Tools

# Accept licences
flutter doctor --android-licenses

# Apple Silicon: emulator is ARM64 - fast, no driver
```

Windows • PowerShell

```
# Install Android Studio (developer.android.com/studio)
# First-run wizard: Standard install.

# SDK Manager → SDK Tools, tick:
#   Android SDK Build-Tools
#   Android SDK Command-line Tools (latest)
#   Android Emulator
#   Android SDK Platform-Tools
#   Android Emulator Hypervisor Driver (AMD / Intel)
#   -- OR -- enable Windows Hypervisor Platform:
#   Control Panel > Programs > Turn Windows features on
#   > Windows Hypervisor Platform (reboot)

# Accept licences (new PowerShell)
flutter doctor --android-licenses
```

Corporate laptops: if Hyper-V / virtualisation is disabled by BIOS or policy the emulator will not boot. Fallback = physical phone with USB debugging, or flutter run -d chrome for UI-only labs.

flutter doctor - read it, fix it, repeat

Green ticks for Flutter, Android toolchain, and one IDE are required. Chrome and Visual Studio warnings can be ignored today.

macOS • zsh

```
flutter doctor
flutter doctor -v          # verbose: paths & versions

# Typical fixes
flutter doctor --android-licenses
sudo xcodebuild -license accept      # iOS only
sudo xcode-select -s /Applications/Xcode.app
flutter config --android-sdk ~/Library/Android/sdk

# Upgrade / channel
flutter upgrade
flutter channel stable
```

Windows • PowerShell

```
flutter doctor
flutter doctor -v          # verbose: paths & versions

# Typical fixes
flutter doctor --android-licenses
# "cmdline-tools component is missing"
#   → SDK Manager > SDK Tools > Android SDK
#       Command-line Tools (latest)
flutter config --android-sdk `
    "$env:LOCALAPPDATA\Android\Sdk"

# "Unable to find git in your PATH" → install Git
#   git-scm.com, tick "Git from the command line"
flutter upgrade
```

Expected on Windows today: [!] Visual Studio not installed (only for Windows desktop apps) - ignore. [!] Chrome - only needed for web.

Create and start an emulator

One Pixel-class device, latest stable API, started once from Device Manager so it is cached for the rest of the week.

macOS • zsh

```
# GUI: Android Studio → Device Manager → Create Device
#   Pixel 8 → latest stable API (arm64-v8a) → Finish → 

# CLI alternative - sdkmanager is DEPRECATED, the GUI route
# above is
# the reliable one. See Appendix E.
sdkmanager "system-images;android-36;google_apis;arm64-v8a"
avdmanager create avd -n pixel8 -k \
  "system-images;android-36;google_apis;arm64-v8a" -d pixel_8

# List and launch
flutter emulators
flutter emulators --launch pixel8
flutter devices
```

Windows • PowerShell

```
# GUI: Android Studio → Device Manager → Create Device
#   Pixel 8 → latest stable API (x86_64) → Finish → 

# CLI alternative - sdkmanager is DEPRECATED, the GUI route
# above is
# the reliable one. See Appendix E.
sdkmanager "system-images;android-36;google_apis;x86_64"
avdmanager create avd -n pixel8 -k `
  "system-images;android-36;google_apis;x86_64" -d pixel_8

# List and launch
flutter emulators
flutter emulators --launch pixel8
flutter devices
```

Emulator hangs on the boot logo → acceleration is missing (previous slide). Emulator boots but is very slow → allocate more RAM in Device Manager > Edit > Show Advanced Settings, or use a physical phone.

Run on a physical Android phone

Faster than an emulator and the fallback for any machine that cannot virtualise.

macOS • zsh

```
# On the phone
#   Settings → About → tap Build number 7× (Developer)
#   Developer options → USB debugging ON
# Plug in, accept the "Allow USB debugging" prompt

adb devices          # should list the phone
flutter devices
flutter run -d <deviceId>

# Wireless (Android 11+)
#   Developer options → Wireless debugging → Pair
adb pair 192.168.0.42:37123
adb connect 192.168.0.42:42001
```

Windows • PowerShell

```
# On the phone
#   Settings → About → tap Build number 7× (Developer)
#   Developer options → USB debugging ON
# Plug in, accept the "Allow USB debugging" prompt

adb devices          # should list the phone
flutter devices
flutter run -d <deviceId>

# Phone not listed? Install the OEM USB driver:
#   Samsung → Smart Switch; Google → Google USB Driver
#   (SDK Manager > SDK Tools); Huawei → HiSuite
# adb is at:
#   $env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe
```

Windows often needs a vendor USB driver before adb sees the phone. macOS needs nothing. Keep one spare USB-C cable at the front; charging cables without data lines are the #1 cause of "not detected".

VS Code setup (what we use all week)

Extensions: Flutter (installs Dart). Optional: Error Lens, GitLens, Flutter Widget Snippets.

Command Palette (Ctrl/Cmd+Shift+P): "Flutter: New Project", "Flutter: Select Device", "Flutter: Open DevTools".

Run: F5 starts debugging with hot reload on save. The status bar bottom-right picks the device.

Refactors: Ctrl/Cmd+. on any widget → "Wrap with Padding / Center / Column", "Extract Widget". You will use this constantly.

Settings worth turning on: "dart.previewFlutterUiGuides", format on save, "dart.lineLength": 100.

```
// .vscode/settings.json (project)
{
  "editor.formatOnSave": true,
  "dart.lineLength": 100,
  "dart.previewFlutterUiGuides": true,
  "dart.flutterHotReloadOnSave": "all",
  "[dart]": {
    "editor.rulers": [100],
    "editor.selectionHighlight": false,
    "editor.suggestSelection": "first",
    "editor.tabCompletion": "onlySnippets",
    "editor.wordBasedSuggestions": "off"
  }
}

// Handy snippets (type + Tab)
//   stless → StatelessWidget
//   stful  → StatefulWidget
```

settings.json

Anatomy of a Flutter project

lib/ all your Dart. `main.dart` is the entry point. Everything else is yours to structure.

pubspec.yaml name, version, dependencies, assets, fonts. The `package.json` / `.csproj` of Flutter.

android/ • ios/ real native projects. You edit them for app id, permissions, signing, flavours (Day 5).

test/ unit and widget tests (Day 5). `integration_test/` for on-device tests.

build/ • .dart_tool/ generated. Git-ignored. Delete with `flutter clean` if things get weird.

web/ • macos/ • windows/ • linux/ other targets; delete or ignore if unused.

```
quote_app/
├── lib/
│   └── main.dart
├── test/
│   └── widget_test.dart
├── android/
│   ├── app/build.gradle(.kts)
│   └── app/src/main/AndroidManifest.xml
├── ios/
│   └── Runner.xcodeproj
├── web/ macos/ windows/ linux/
├── pubspec.yaml      ← dependencies, assets
├── pubspec.lock
├── analysis_options.yaml ← lint rules
└── .dart_tool/ build/  ← generated
```

tree

Flutter CLI essentials

Command	What it does	When
<code>flutter create quote_app</code>	New project with Android/iOS/web targets	Once
<code>flutter create --org za.co.moint quote_app</code>	Sets the package / bundle id from the start	Prefer this
<code>flutter run</code> · <code>flutter run -d chrome</code>	Build + run on selected device; r reload, R restart, q quit	Every lab
<code>flutter pub add dio</code> · <code>flutter pub get</code>	Add a dependency / install from pubspec	Adding packages
<code>flutter pub outdated</code> · <code>flutter pub upgrade</code>	Show / apply newer versions	Weekly
<code>flutter analyze</code>	Static analysis with lint rules	Before commit, in CI
<code>flutter test</code>	Run unit + widget tests	Day 5, CI
<code>flutter clean</code>	Delete build outputs	When builds misbehave
<code>flutter build apk</code> / <code>appbundle</code> / <code>ipa</code>	Release artefacts	Day 5
<code>dart format .</code> · <code>dart fix --apply</code>	Format code / apply automated fixes	Any time

Hot reload vs hot restart vs full rebuild

Hot reload • r • under 1 s

- Injects changed Dart into the running VM
- Keeps app state (navigation stack, form values)
- Works for: widget build methods, most logic
- Does NOT pick up: changes to main(), global initialisers, enum values, const values, generic type changes

Hot restart • R • a few seconds

- Re-runs main() from scratch on the same build
- State is lost
- Use when reload "did nothing" or after changing initialisation code
- Full rebuild (stop, flutter run) only for: new plugins with native code, pubspec changes to native deps, AndroidManifest / Info.plist edits

Rule of thumb: try r; if the change is not visible, R; if it still fails or you added a plugin, stop and run again.

Windows room, macOS presenter: the differences that matter

Paths: SDK at C:\src\flutter vs ~/development/flutter. Pub cache at %LOCALAPPDATA%\Pub\Cache vs ~/.pub-cache. Android SDK at %LOCALAPPDATA%\Android\Sdk vs ~/Library/Android/sdk.

Shell: PowerShell line continuation is a backtick (`), zsh is a backslash. \$env:VAR vs \$VAR. Quoting differs.

Emulator: Windows needs a hypervisor driver; Apple Silicon does not.

USB devices: Windows usually needs a vendor driver; macOS does not.

iOS: cannot be built on Windows at all. Friday: iOS demo from the Mac + Codemagic cloud build for the team.

Line endings: set git config --global core.autocrlf true on Windows so Dart files do not churn.

Every command slide in this course has a macOS column and a Windows column. Appendix A has the full path and command map.

Goal Every machine runs the template app on an emulator or phone with working hot reload.

1. FIRST, BEFORE ANYTHING ELSE - three SEPARATE lines (Windows PowerShell 5.1 has no &&): flutter create --org za.co.moint quote_app / cd quote_app / flutter build apk --debug - then leave it running while the instructor teaches. It downloads ~4 GB (NDK 2.8 GB, SDK Platform, Gradle) and takes 7-8 minutes.
2. While that runs: flutter doctor → resolve every red X (Windows column on the slides; ask if stuck more than 5 min).
3. Start your emulator (or plug in a phone) → flutter devices shows it.
4. When the build finishes: flutter run → this one is fast because the first build warmed everything.
5. Delete the generated test that still references MyApp: del test\widget_test.dart (macOS: rm test/widget_test.dart) - we write a real one on Day 5. Without this, flutter analyze fails and dart fix will NOT repair it.
6. Change the AppBar title text in lib/main.dart, save → press r → see it update without losing the counter value.
7. Press R and confirm the counter resets. Press q to quit.
8. Open VS Code in the folder, press F5, confirm the debugger attaches.
9. Commit, one line at a time (no && in PowerShell 5.1): git init / git add . / git commit -m "Day 1: template app"

Stretch (fast lane)

- Add the Error Lens extension and break a line on purpose; read the inline error.
- Run the app in Chrome: flutter run -d chrome. Note what looks different.
- Look inside android/app/build.gradle(.kts) and find applicationId - we change it on Day 5.

MODULE 2 • 10:45 - 12:30

Dart primer for experienced developers

By the end of this module you can

- Read and write null-safe Dart fluently
- Use named parameters, final/const, and constructors idiomatically
- Model domain data with classes, records, enums and sealed types
- Recognise the five Dart habits that differ from TypeScript, C# and Java

Dart in one table (vs TypeScript, C#, Java)

Concept	TypeScript	C#	Dart
Immutable binding	<code>const x = 1</code>	<code>readonly / const</code>	<code>final x = 1; (runtime) · const x = 1; (compile-time)</code>
Nullable	<code>string null</code> (optional check)	<code>string?</code> (warnings)	<code>String?</code> - enforced at compile AND runtime
Named args	Object destructuring	Named arguments	<code>f({required int age, bool x = true})</code>
Class fields via ctor	Parameter properties	Primary constructors	<code>Quote({required this.id})</code>
Interfaces	<code>interface</code>	<code>interface</code>	Every class is an interface: implements
Mixins	-	-	<code>mixin Loggable {} · class A with Loggable</code>
Records / tuples	Tuple types	Tuples / records	<code>(double, String) · ({int a, int b})</code>
Discriminated unions	Union + kind	Abstract records + switch	<code>sealed class + switch expression</code>
Async	<code>Promise<T></code>	<code>Task<T></code>	<code>Future<T></code>
Streams	<code>Observable (rxjs)</code>	<code>IAsyncEnumerable<T></code>	<code>Stream<T></code>
Packages	npm / package.json	NuGet / .csproj	pub / pubspec.yaml

Variables: var, final, const, late

var type is inferred once, then fixed. `var n = 1; n = "x";` is a compile error.

final assigned once, at runtime. This is JS `const`. Use it by default.

const compile-time constant. Deep-immutable. Also used on widget constructors for performance (later).

late "I promise to assign this before reading it." Non-null without an initial value. Throws `LateInitializationError` if you lie.

Type annotations are optional but idiomatic on fields and public APIs; inference inside functions.

```
var count = 0;           // int, mutable
count = 5;               // ok
// count = 'five';      // error: String is not int

final createdAt = DateTime.now(); // set once, at runtime
// createdAt = DateTime.now();    // error

const vatRate = 0.15;      // compile-time
const brand = 'Alpha Insure';

late String token;         // assigned later, non-null
void init() { token = 'abc'; }
// print(token) before init() → LateInitializationError

// Explicit types where it helps readers
double premium = 1450.0;
List<String> makes = ['Toyota', 'VW'];
```

DartPad

Null safety, part 1: the type is the contract

String can never be null. The compiler refuses to assign null.

String? may be null. You must check before using it as a String.

?. null-aware call; **??** default; **??=** assign if null. Same as TS/C#.

! the bang: "trust me, it is not null". Throws at runtime if wrong. Treat every **!** as a code smell to justify.

Flow analysis promotes a local after a null check, exactly like TS narrowing. Fields are NOT promoted (next slide).

```
String name = 'Sive';  
// name = null;           // compile error  
  
String? nickname;         // starts null  
print(nickname?.length);  // null  
print(nickname ?? 'none'); // 'none'  
nickname ??= 'S';         // assign if null  
  
int len = nickname!.length; // ok now, throws if null  
  
// Promotion on locals  
String? maybe = fetch();  
if (maybe != null) {  
  print(maybe.length);    // maybe is String here  
}  
// Early return also promotes  
if (maybe == null) return;  
print(maybe.toUpperCase()); // String
```

DartPad

Null safety, part 2: fields, collections, and the patterns that fix them

Fields do not promote. `this.nickname != null` does not narrow - another method could change it. Copy to a local first, or use `?.` / `??`.

Collections: `List<String?>` is a list that may contain nulls; `List<String>?` is a list that may itself be null. Different things.

required on a named parameter replaces "non-optional". Non-nullable named params must be required or have a default.

Avoid: `dynamic` (opts out of typing) and `!` (opts out of safety). Prefer `?.`, `??`, if-null returns, and `required`.

```
class Driver {
  String? nickname;
  void greet() {
    // if (nickname != null) print(nickname.length); // ERROR
    final n = nickname;           // copy to local
    if (n != null) print(n.length); // promoted
    print(nickname?.length ?? 0);  // or null-aware
  }
}

List<String?> a = ['x', null]; // items may be null
List<String>? b;               // list may be null
b?.first;                      // guard the list

double quote({required int age, int? claims}) {
  final c = claims ?? 0;
  return age < 25 ? 2000 + c * 150 : 1200 + c * 100;
}
quote(age: 30);                // claims omitted → null
```

DartPad

Why fields don't promote - read it, then teach it back

dart.dev/null-safety/understanding-null-safety

Sections: "Flow analysis" and "Working with nullable fields" (about 8 minutes)

Answer these as you read

1. In one sentence: why can a local variable be promoted after a null check but a field cannot?
2. What are the three idioms the doc recommends for nullable fields? Which one will you use most?
3. What does the "!" operator actually do at runtime, and what does the doc say about when it is acceptable?
4. Find one sentence in the doc you disagree with or would phrase differently for your team.

Then

- One volunteer explains Q1 to the room in under 60 seconds, no slides.
- Someone else adds what they would change (Q4).
- Instructor fills gaps only after both have spoken.

Functions: positional, optional, named, arrow

Positional f(a, b). Optional positional in square brackets: f(a, [b]).

Named f({required a, b = 1}). Called as f(a: 1). Any order. This is how every widget constructor works.

Arrow => for single-expression bodies. Same as JS/C# lambdas.

Functions are values pass them, store them, return them. Type: int Function(String).

Closures capture variables like JS. Used for every onPressed callback.

```
// positional + optional positional
String label(String make, [String? model]) =>
  model == null ? make : '$make $model';

// named with required and defaults
double premium(double base,
  {required int age, bool comprehensive = true}) {
  final loading = age < 25 ? 1.5 : 1.0;
  return comprehensive ? base * 1.4 * loading : base * loading;
}
premium(1000, age: 34); // 1400.0
premium(1000, age: 22, comprehensive: false); // 1500.0

// function types & closures
final int Function(int) twice = (x) => x * 2;
List<int> Function() counter() {
  var n = 0;
  return () => [n++];
}
void Function() onTap = () => print('tapped');
```

DartPad

Classes and constructors

Field-initialising ctor `Quote({required this.id})` assigns fields directly. No boilerplate body.

const constructor all fields final → instances can be compile-time constants. Widgets rely on this.

Named constructors `Quote.empty()`, `Quote.fromJson(...)` - multiple ways to build one type.

factory a constructor that may return an existing/cached/sub-type instance; the `fromJson` convention.

Getters/setters look like fields to callers. No parentheses.

```
class Quote {  
  final String id;  
  final double premium;  
  final String currency;  
  
  const Quote({  
    required this.id,  
    required this.premium,  
    this.currency = 'ZAR',  
  });  
  
  // named constructor  
  const Quote.empty() : id = '', premium = 0, currency = 'ZAR';  
  
  // factory - the fromJson convention  
  factory Quote.fromJson(Map<String, dynamic> j) => Quote(  
    id: j['id'] as String,  
    premium: (j['premium'] as num).toDouble(),  
    currency: j['currency'] as String? ?? 'ZAR',  
  );  
  
  Map<String, dynamic> toJson() =>  
    {'id': id, 'premium': premium, 'currency': currency};  
  
  String get display => '$currency ${premium.toStringAsFixed(2)}';  
}
```

DartPad

Immutability: copyWith, ==, and when to reach for a package

Dart objects are mutable by default but Flutter state works best with immutable models and new copies.

copyWith the idiom for "same object, one field changed". Null means "keep".

Equality default == is identity. Override == and hashCode for value equality, or use the equatable package.

freezed code-generates copyWith, ==, toString, JSON and union types. Popular in production; we show it on Day 4.

```
class QuoteRequest {  
  final String make;  
  final int year;  
  final Cover cover;  
  const QuoteRequest(  
    {required this.make, required this.year, required this.cover});  
  
  QuoteRequest copyWith({String? make, int? year, Cover? cover}) =>  
    QuoteRequest(  
      make: make ?? this.make,  
      year: year ?? this.year,  
      cover: cover ?? this.cover,  
    );  
  
  @override  
  bool operator ==(Object o) =>  
    o is QuoteRequest &&  
    o.make == make && o.year == year && o.cover == cover;  
  
  @override  
  int get hashCode => Object.hash(make, year, cover);  
}  
  
final r1 = const QuoteRequest(make: 'VW', year: 2020, cover:  
  Cover.comprehensive);  
final r2 = r1.copyWith(year: 2021);    // r1 untouched
```

DartPad

Inheritance, abstract classes, interfaces, mixins

extends single inheritance, super calls, @override. Same as C#/Java.

abstract class cannot be instantiated; may have implemented members.

implements any class doubles as an interface; you must implement every member. abstract interface class is the explicit form.

mixin ... with reusable behaviour without inheritance. Order matters: last wins.

extension add methods to existing types, even ones you do not own.

```
abstract interface class QuoteService {
  Future<Quote> getQuote(QuoteRequest r);
}

mixin Loggable {
  void log(String m) => print('[${runtimeType}] $m');
}

class FakeQuoteService with Loggable implements QuoteService {
  @override
  Future<Quote> getQuote(QuoteRequest r) async {
    log('quoting ${r.make}');
    await Future.delayed(const Duration(seconds: 1));
    return Quote(id: 'q1', premium: r.year < 2015 ? 1800 : 1200);
  }
}

class Vehicle { const Vehicle(this.make); final String make; }
class Car extends Vehicle {
  const Car(super.make, this.doors);
  final int doors;
}

extension Money on double {
  String get rands => 'R ${toStringAsFixed(2)}';
}
print(1450.0.rands);    // R 1450.00
```

DartPad

Collections: List, Set, Map, and collection expressions

Literals [...] List, {...} Set, {k: v} Map. Generic types inferred or explicit.

Spread and if/for inside literals build lists declaratively - used constantly in widget children.

Iterable is lazy .map() and .where() return Iterables; call .toList() when you need a List. #1 beginner bug.

Common methods map, where, firstWhere, any, every, fold, reduce, sorted (via sort), join, take, skip.

```
final makes = <String>['Toyota', 'VW', 'BMW'];
final years = {2019, 2020, 2020}; // Set → {2019, 2020}
final byMake = {'VW': 1200.0, 'BMW': 2100.0};

// collection-if, collection-for, spread
final showLuxury = true;
final list = [
  'Toyota',
  if (showLuxury) 'BMW',
  for (final m in makes) m.toUpperCase(),
  ...makes,
];

// lazy iterables - remember toList()
final cheap = byMake.entries
  .where((e) => e.value < 2000)
  .map((e) => e.key)
  .toList(); // ['VW']

final total = byMake.values.fold<double>(0.0, (a, b) => a + b);
final first = makes.firstWhere((m) => m.startsWith('B'),
  orElse: () => 'none');
makes.sort(); // in place
```

DartPad

Control flow and switch expressions

if / for / while identical to C-family. for-in over Iterables.

switch statement no fall-through needed; break is implicit.

switch expression (Dart 3) returns a value; must be exhaustive; uses patterns. This is the one to learn.

Exhaustiveness on enums and sealed classes the compiler errors if you miss a case. Add a case, every switch in the app flags itself.

```
enum Cover { thirdParty, thirdPartyFireTheft, comprehensive }

// switch statement
String describe(Cover c) {
  switch (c) {
    case Cover.thirdParty:      return 'TP';
    case Cover.thirdPartyFireTheft: return 'TPFT';
    case Cover.comprehensive:   return 'Comp';
  }
}

// switch EXPRESSION - preferred
double factor(Cover c) => switch (c) {
  Cover.thirdParty      => 0.6,
  Cover.thirdPartyFireTheft => 0.8,
  Cover.comprehensive    => 1.0,
};

// guards and ranges
String band(int age) => switch (age) {
  < 18      => 'not eligible',
  >= 18 && < 25 => 'young driver',
  _         => 'standard',
};
```

DartPad

Records: lightweight tuples with names

Positional record (double, String) - access with \$1, \$2.

Named record ({double amount, String currency}) - access by name.

Destructuring final (a, b) = f(); final (:amount, :currency) = g();

Use for returning two values, small ad-hoc groupings, keys. Not for domain models (use classes).

Value equality is built in (1, 2) == (1, 2) is true.

```
// return two values without a class
(double, String) priceWithCurrency(Quote q) =>
  (q.premium, q.currency);

final (amount, currency) = priceWithCurrency(quote);
print('$currency $amount');

// named record
({double min, double max}) range(List<double> xs) =>
  (min: xs.reduce((a, b) => a < b ? a : b),
   max: xs.reduce((a, b) => a > b ? a : b));

final r = range([1200, 1800, 950]);
print(r.min); // 950.0
final (:min, :max) = r;

// records as map keys / in sets - value equality
final seen = <(String, int)>{};
seen.add(('VW', 2020));
print(seen.contains(('VW', 2020))); // true
```

DartPad

Sealed classes + patterns = discriminated unions

sealed class all subtypes must be declared in the same library. The compiler knows the full set.

switch is exhaustive miss a subtype → compile error. No default branch needed or wanted.

Object patterns `Loaded(:final quote)` destructures fields in the case.

This is how we model UI state Idle / Loading / Loaded / Failed. Day 3 builds on exactly this.

```
sealed class QuoteState { const QuoteState(); }
class QuoteIdle extends QuoteState { const QuoteIdle(); }
class QuoteLoading extends QuoteState { const QuoteLoading(); }
class QuoteLoaded extends QuoteState {
  const QuoteLoaded(this.quote);
  final Quote quote;
}
class QuoteFailed extends QuoteState {
  const QuoteFailed(this.message);
  final String message;
}

String describe(QuoteState s) => switch (s) {
  QuoteIdle() => 'Fill in the form',
  QuoteLoading() => 'Calculating...',
  QuoteLoaded(:final quote) => 'Premium ${quote.display}',
  QuoteFailed(:final message) => 'Error: $message',
};

// Add a subtype tomorrow → every switch fails to compile
// until you handle it. That is the point.
```

DartPad

Five Dart habits that surprise TS / C# / Java developers

Habit	What happens	Do this instead
Forgetting <code>.toList()</code>	<code>map/where</code> return lazy Iterables; type errors at children:	Chain <code>.toList()</code> at the end, or use <code>collection-for</code>
Using <code>!</code> to silence the compiler	Runtime null crash later	<code>?.</code> , <code>??</code> , early return, <code>required</code> , or copy field to a local
Mutating state objects	Widgets don't rebuild; Riverpod/Bloc don't notify	Immutable models + <code>copyWith</code> ; assign a new object
<code>==</code> on your own classes	Identity comparison; two equal quotes are "different"	Override <code>==/hashCode</code> , or <code>equatable</code> / <code>freezed</code>
Missing <code>const</code>	Widgets rebuild needlessly; lint warns	Add <code>const</code> where the analyser suggests (<code>dart fix --apply</code>)
Expecting <code>int/double</code> interchange	<code>1</code> is <code>int</code> , <code>1.0</code> is <code>double</code> ; JSON numbers vary	Cast through <code>num</code> : <code>(j['x'] as num).toDouble()</code>
<code>print</code> for debugging	Works, but noisy in release	<code>debugPrint</code> , or the <code>logger</code> package

Quote domain model in DartPad

Goal A null-safe, immutable domain model for the quote app, exercised from `main()`. Ready-to-paste versions of every code block today are in the repo under `dartpad/` - open the file, select all, paste into `dartpad.dev`, Run.

1. Open `dartpad.dev` (New Pad → Dart). Everything today runs in the browser; no SDK needed.
2. Create enum `Cover` { `thirdParty`, `thirdPartyFireTheft`, `comprehensive` } with a double get factor via switch expression (0.6 / 0.8 / 1.0).
3. Create class `QuoteRequest`: `make`, `model` (nullable), `year`, `driverAge`, `cover`. `const` constructor, required named params, `copyWith`.
4. Create class `Quote`: `id`, `premium`, `currency` = "ZAR", a display getter, `fromJson` / `toJson`.
5. Write `double calculatePremium(QuoteRequest r)`: base 1000; $\times 1.5$ if `driverAge < 25`; $\times 1.2$ if `year < 2015`; $\times$ `cover.factor`. Round to 2 decimals.
6. Create the sealed `QuoteState` (`Idle` / `Loading` / `Loaded` / `Failed`) and `String describe(QuoteState)` with an exhaustive switch expression.
7. In `main()`: build three requests with `copyWith`, print each premium, then print `describe()` for all four states.
8. Save the pad URL - you will paste this code into the app in Lab 1.4.

Stretch (fast lane)

- Override `==` and `hashCode` on `QuoteRequest` and prove `r1.copyWith() == r1`.
- Add `QuoteExpired` to the sealed class and watch `describe()` fail to compile until you handle it.
- Add an extension `Money` on `double` with a `rand`s getter and use it in `Quote.display`.

MODULE 3 • 13:15 - 14:30

Async Dart

By the end of this module you can

- Map Future to Promise / Task and Stream to Observable
- Write async / await with proper error handling and timeouts
- Run work in parallel with Future.wait
- Produce and consume Streams; know when an Isolate is needed

Future<T> is Promise<T> / Task<T>

TypeScript

```
async function getQuote(id: string): Promise<Quote> {
  const res = await fetch(`/quotes/${id}`);
  if (!res.ok) throw new Error('HTTP ' + res.status);
  return (await res.json()) as Quote;
}

try {
  const q = await getQuote('q1');
} catch (e) {
  console.error(e);
} finally {
  setLoading(false);
}

const all = await Promise.all([a(), b()]);
```

Dart

```
Future<Quote> getQuote(String id) async {
  final res = await dio.get('/quotes/$id');
  if (res.statusCode != 200) throw Exception('HTTP');
  return Quote.fromJson(res.data);
}

try {
  final q = await getQuote('q1');
} on DioException catch (e) {
  debugPrint('network: ${e.message}');
} catch (e, st) {
  debugPrint('$e\n$st');
} finally {
  setLoading(false);
}

final all = await Future.wait([a(), b()]);
```

Same shape. Differences: typed catch with "on", stack trace as a second catch parameter, Future.wait instead of Promise.all. An un-awaited Future still runs (no lazy evaluation).

async / await details you will hit this week

Every async function returns a Future even void ones: `Future<void>`. Mark them so callers can await.

Timeouts `.timeout(Duration)` throws `TimeoutException` - always set one on network calls.

Sequential vs parallel two awaits in a row are sequential; `Future.wait` runs them together.

Error propagation throw inside async → rejected Future; unhandled → crashes in debug, logged in release.

In widgets after an await, check if `(!context.mounted)` return; before touching UI. Tomorrow.

```
Future<void> submit(QuoteRequest r) async {
  try {
    final quote = await service
      .getQuote(r)
      .timeout(const Duration(seconds: 8));
    print(quote.display);
  } on TimeoutException {
    print('Slow network, try again');
  }
}

// sequential (2 s total)
final a = await slow(1); final b = await slow(1);

// parallel (1 s total)
final (x, y) = await (slow(1), slow(1)).wait; // Dart 3 records
// or: final rs = await Future.wait([slow(1), slow(1)]);

// fire-and-forget on purpose
unawaited(analytics.log('quote_started'));

// Future chaining still exists, prefer await
service.getQuote(r).then(print).catchError(print);
```

DartPad

Stream<T> is Observable<T> / IAsyncEnumerable<T>

A sequence of async values ticks, socket messages, database changes, auth state.

Produce `async*` + `yield`, or a `StreamController`.

Consume `await for`, or `.listen((v) {...})` which returns a subscription you must cancel.

Transform `map`, `where`, `take`, `distinct`, `asyncMap`, `debounce` (via `rxdart`).

Single- vs broadcast default streams allow one listener; `.asBroadcastStream()` for many.

```
// produce
Stream<int> countdown(int from) async* {
  for (var i = from; i >= 0; i--) {
    await Future.delayed(const Duration(seconds: 1));
    yield i;
  }
}

// consume - await for
await for (final n in countdown(3)) print(n);

// consume - listen (remember to cancel)
final sub = countdown(3).listen(print);
await Future.delayed(const Duration(seconds: 2));
await sub.cancel();

// controller: push values manually
final ctrl = StreamController<String>.broadcast();
ctrl.stream.where((s) => s.isNotEmpty).listen(print);
ctrl.add('quote_started');
await ctrl.close();

// Flutter: StreamBuilder<T>(stream: ..., builder: ...)
// Riverpod: StreamProvider (Day 3/4)
```

DartPad

Isolates: when async is not enough

Dart is single-threaded async/await interleaves I/O; it does not run CPU work in parallel.

Heavy CPU work blocks the UI parsing a 5 MB JSON, image processing, big loops → dropped frames.

Isolate a separate memory heap + event loop. No shared state; messages are copied.

compute() Flutter helper: run a top-level function in an isolate and get the result as a Future.

Rule if it takes more than ~16 ms of CPU, move it off the main isolate.

```
import 'package:flutter/foundation.dart';  
  
// must be a top-level or static function  
List<Quote> parseQuotes(String body) {  
  final list = jsonDecode(body) as List;  
  return list.map((e) => Quote.fromJson(e)).toList();  
}  
  
Future<List<Quote>> load(String body) =>  
  compute(parseQuotes, body); // runs off the UI thread  
  
// Dart 2.19+: Isolate.run  
final quotes = await Isolate.run(() => parseQuotes(body));
```

Dart

Network I/O is already non-blocking. Isolates are for CPU, not for waiting.

Goal A fake QuoteService with realistic latency, failure, timeout, and a Stream of quote states.

1. In your Lab 1.2 pad, add abstract interface class QuoteService { Future<Quote> getQuote(QuoteRequest r); }
2. Implement FakeQuoteService: await Future.delayed(1500 ms); throw Exception("Vehicle too old") if year < 2000; else return Quote with calculatePremium.
3. Write Future<void> runOnce(QuoteRequest r): try/on/catch with .timeout(3 s); print the result or the error.
4. Write Stream<QuoteState> quoteStates(QuoteService s, QuoteRequest r) async*: yield Loading, then Loaded or Failed.
5. In main(): await for over quoteStates(...) and print describe(state) for a valid and an invalid request.
6. Run three requests in parallel with Future.wait and print how long they took (Stopwatch).

Stretch (fast lane)

- Make FakeQuoteService fail randomly 30% of the time and add one retry with exponential back-off.
- Add a Stream transformer that debounces repeated identical requests (hint: distinct()).

MODULE 4 · 14:45 - 16:00 (theory to 15:15 · lab
15:15-15:55 · recap 15:55)

Your first Flutter app

By the end of this module you can

- Explain the widget tree and the three trees behind it
- Read main.dart, MaterialApp and Scaffold line by line
- Choose between StatelessWidget and StatefulWidget and use setState correctly
- Use DevTools and the Widget Inspector
- Leave with the quote app skeleton running

Everything is a widget

A widget is an immutable description of a piece of UI: what to show and how to configure children.

Visible widgets Text, Icon, Image, ElevatedButton, TextField.

Layout widgets Row, Column, Padding, Center, Expanded, SizedBox. They draw nothing themselves.

Behaviour widgets GestureDetector, Theme, MediaQuery, Navigator - wrap children to add behaviour or context.

Composition over inheritance you build UIs by nesting, not by subclassing Button.

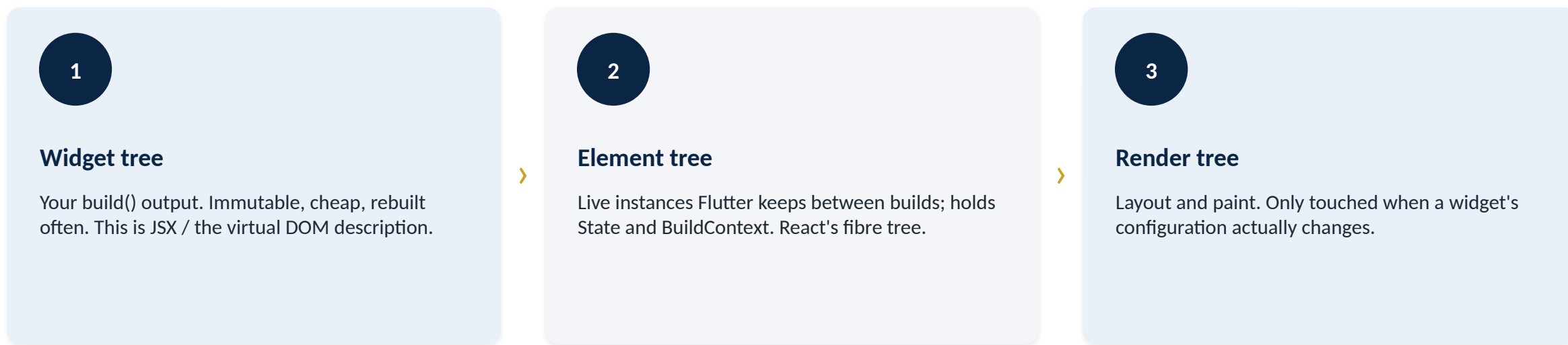
```
// React
<View style={{padding: 16}}>
  <Text style={{fontSize: 24}}>Premium</Text>
  <Pressable onPress={go}><Text>Accept</Text></Pressable>
</View>

// Flutter - layout and style are widgets too
Padding(
  padding: const EdgeInsets.all(16),
  child: Column(
    children: [
      const Text('Premium', style: TextStyle(fontSize: 24)),
      FilledButton(onPressed: go, child: const Text('Accept')),
    ],
  ),
)
```

Dart

Deep nesting is normal. VS Code: Ctrl/Cmd+. → "Extract Widget" keeps it readable.

Three trees: widget → element → render object



You write widgets only. Flutter diffs each new widget tree against the element tree by type and key, updates in place where they match, and re-creates only where they differ - the same reconciliation idea as React. BuildContext is a widget's element: its position in the tree, used to look up ancestors like Theme.of(context).

The three trees - draw it together (5 min)

`docs.flutter.dev/resources/inside-flutter`

Instructor-led whiteboard sketch. This page is OPTIONAL reading tonight, alongside the constraints doc.

Answer these as you read

1. Why is it cheap to create thousands of widget objects per frame? What keeps them cheap?
2. What does Flutter compare when it decides whether to reuse an Element - and where do Keys come in?
3. Which tree holds State, and why does that mean a StatefulWidget keeps its counter across rebuilds?
4. Map it to your framework: React fibre / Angular component instance - where does each tree land?

Then

- YOU draw widget → element → render for the counter app while asking the four questions.
- Take answers from the room; do not wait for silence.
- Photograph the whiteboard. Inside Flutter is optional reading tonight.

main.dart, line by line

```
import 'package:flutter/material.dart';    // Material widgets, Theme, Scaffold...

void main() {
  runApp(const QuoteApp());                // inflate the root widget and attach to the screen
}

class QuoteApp extends StatelessWidget {    // no mutable state → Stateless
  const QuoteApp({super.key});             // const ctor; key forwarded to the framework

  @override
  Widget build(BuildContext context) {      // called whenever Flutter needs this widget's UI
    return MaterialApp(                   // app-wide: theme, navigation, localisation
      title: 'Quote App',
      theme: ThemeData(colorSchemeSeed: const Color(0xFF0B2545), useMaterial3: true),
      home: const HomeScreen(),           // first screen
    );
  }
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});
  @override
  Widget build(BuildContext context) => Scaffold( // page chrome: app bar, body, FAB, drawer
    appBar: AppBar(title: const Text('Get a quote')),
    body: const Center(child: Text('Hello, Flutter')),
  );
}
```

lib/main.dart

main() → runApp → MaterialApp → Scaffold → your widgets. MaterialApp is <App> + router + theme provider in one; Scaffold is the page frame.

StatelessWidget vs StatefulWidget

StatelessWidget - pure function of its inputs

```
class PremiumBadge extends StatelessWidget {
  const PremiumBadge({super.key, required this.amount});
  final double amount;

  @override
  Widget build(BuildContext context) {
    final style = Theme.of(context).textTheme.headlineMedium;
    return Text('R ${amount.toStringAsFixed(0)}', style:
style);
  }
}

// Rebuilt when the parent passes new inputs.
// No internal mutable state. Prefer this.
```

StatefulWidget - owns mutable state

```
class CoverPicker extends StatefulWidget {
  const CoverPicker({super.key});
  @override
  State<CoverPicker> createState() => _CoverPickerState();
}

class _CoverPickerState extends State<CoverPicker> {
  Cover _cover = Cover.comprehensive; // the state

  @override
  Widget build(BuildContext context) {
    return SegmentedButton<Cover>(
      segments: [for (final c in Cover.values)
        ButtonSegment(value: c, label: Text(c.name))],
      selected: {_cover},
      onSelectionChanged: (s) => setState(() => _cover =
s.first),
    );
  }
}
```

Two classes for one stateful widget: the Widget (immutable config, recreated on every parent build) and the State (persists in the element tree). `setState(() {...})` mutates the state and schedules a rebuild of this subtree only.

StatefulWidget lifecycle



React mapping: `initState` ≈ `useEffect(() => ..., [])` mount; `dispose` ≈ its cleanup; `didUpdateWidget` ≈ effect with a dependency; `build` ≈ render. Rule for `build()`: no network calls, no `setState`, no heavy work - it can run 60 times a second.

The counter app, annotated

```
class CounterPage extends StatefulWidget {  
  const CounterPage({super.key, required this.title});  
  final String title; // config from parent → widget.title  
  @override  
  State<CounterPage> createState() => _CounterPageState();  
}  
  
class _CounterPageState extends State<CounterPage> {  
  int _counter = 0; // state survives rebuilds  
  
  void _increment() => setState(() => _counter++); // mutate inside setState → rebuild  
  
  @override  
  Widget build(BuildContext context) {  
    final theme = Theme.of(context); // inherited lookup via context  
    return Scaffold(  
      appBar: AppBar(title: Text(widget.title)),  
      body: Center(  
        child: Column(  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: [  
            const Text('You have pushed the button this many times:'),  
            Text('$_counter', style: theme.textTheme.headlineMedium),  
          ],  
        ),  
      ),  
      floatingActionButton: FloatingActionButton(onPressed: _increment, child: const Icon(Icons.add)),  
    );  
  }  
}
```

lib/main.dart

Hot reload (r) keeps `_counter`. Hot restart (R) resets it. Try both now.

Adding a package from pub.dev

pub.dev is npm/NuGet for Dart. Check the Likes, Pub Points, and "Flutter Favorite" badge, then add with the CLI so versions are pinned correctly.

macOS • zsh

```
# add a runtime dependency
flutter pub add intl

# add a dev dependency
flutter pub add --dev mocktail

# add a specific version / constraint
flutter pub add dio:^5.4.0

# after editing pubspec.yaml by hand
flutter pub get

# see what is outdated
flutter pub outdated

# pub cache location
ls ~/.pub-cache
```

Windows • PowerShell

```
# add a runtime dependency
flutter pub add intl

# add a dev dependency
flutter pub add --dev mocktail

# add a specific version / constraint
flutter pub add dio:^5.4.0

# after editing pubspec.yaml by hand
flutter pub get

# see what is outdated
flutter pub outdated

# pub cache location
dir $env:LOCALAPPDATA\Pub\Cache
```

A package with native code (sqflite, camera, path_provider...) needs a full stop-and-run after adding, not hot reload. Symptom if you forget: MissingPluginException.

Debugging: DevTools and the Widget Inspector

Breakpoints F5 in VS Code, click the gutter. Debug console shows print/debugPrint.

Open DevTools VS Code Command Palette → "Flutter: Open DevTools", or the link printed by flutter run.

Widget Inspector click any element on the emulator to jump to its widget in the tree and its source line. Toggle "Select Widget Mode".

Layout Explorer visualises Row/Column flex, constraints and overflow. Tomorrow's best friend.

Performance / Memory / Network tabs Day 5.

debugPrint vs print debugPrint throttles long output on Android and is stripped less noisily.

```
// quick visual aids you can toggle at runtime
import 'package:flutter/rendering.dart';

void main() {
  debugPaintSizeEnabled = true; // draw boxes around every widget
  runApp(const QuoteApp());
}

// in code
debugPrint('state: $state');
assert(premium >= 0, 'premium negative'); // debug-only

// flutter run key shortcuts
// r hot reload      R hot restart
// p toggle debug paint
// o toggle Android/iOS rendering
// w dump widget tree to console
// q quit
```

Dart

Project structure for the quote app (feature-first)

core/ theme, routing, networking, shared widgets. Nothing domain-specific.

features/<name>/ one folder per feature: data (API, storage), domain (models, logic), presentation (screens, widgets, providers).

main.dart bootstrapping only: runApp, ProviderScope (Day 3), router (Day 4).

Why features are deletable and testable in isolation; new developers find things by feature, not by layer.

Today create the folders and move the Lab 1.2 model into features/quote/domain.

```
lib/
├── main.dart
├── app.dart                ← MaterialApp + theme
├── core/
│   ├── theme/brand_theme.dart    (Day 2)
│   ├── routing/router.dart       (Day 4)
│   ├── network/dio_client.dart   (Day 4)
│   └── widgets/                  shared UI
└── features/
    ├── onboarding/
    │   └── presentation/onboarding_screen.dart
    └── quote/
        ├── domain/
        │   ├── quote.dart
        │   ├── quote_request.dart
        │   ├── quote_state.dart
        │   └── premium_calculator.dart
        ├── data/
        │   ├── quote_service.dart    (interface + fake)
        │   └── quote_repository.dart (Day 4)
        └── presentation/
            ├── capture_screen.dart    (Day 2)
            ├── result_screen.dart
            └── quote_provider.dart     (Day 3)
```

tree

Goal The quote app runs on your emulator with the domain model inside it and one stateful screen.

1. In `quote_app`, create the folder structure from the previous slide (empty files are fine for Day 2+ items).
2. Paste your Lab 1.2/1.3 code into `features/quote/domain/*.dart` and `data/quote_service.dart`. Fix imports (`package:quote_app/...`).
3. Create `app.dart`: `QuoteApp` (`StatelessWidget`) returning `MaterialApp`(title, theme with `colorSchemeSeed`, `home: CaptureScreen()`).
4. `main.dart`: `void main() => runApp(const QuoteApp());`
5. `CaptureScreen` (`StatefulWidget`): Scaffold with `AppBar` "Get a quote", a `SegmentedButton` for Cover stored in state, a `FilledButton` "Calculate", and a `Text` showing the premium.
6. On button press: build a `QuoteRequest`(`make: "VW"`, `year: 2020`, `driverAge: 30`, `cover: _cover`) → `calculatePremium` → `setState` the result.
7. Extract the premium `Text` into a `PremiumBadge` `StatelessWidget` using `Ctrl/Cmd+`. → Extract Widget.
8. Open the Widget Inspector, select the badge, confirm it maps to your file. Commit "Day 1: skeleton".

Stretch (fast lane)

- Make the Calculate button `async`: `await FakeQuoteService().getQuote(...)` with a loading indicator; guard with `if (!mounted)` return;.
- Add flutter pub add intl and format the premium with `NumberFormat.currency(locale: "en_ZA", symbol: "R ")`.
- Add a second screen `ResultScreen` and `Navigator.push` to it (preview of Day 4).

Tonight: read the constraints doc - tomorrow starts with your explanation

`docs.flutter.dev/ui/layout/constraints`

Whole page, including all 29 examples (about 25 minutes). Run a few in DartPad.

Answer these as you read

1. Say the rule "constraints go down, sizes go up, parent sets position" in your own words, with one example.
2. Why does a Container with no child fill its parent, but the same Container inside a Column has zero height?
3. What is an "unbounded constraint" and which two widgets most often cause it?
4. Pick one of the 29 examples that surprised you and be ready to show it.

Then

- 09:00 tomorrow: three delegates each explain one question at the whiteboard before any slide goes up.
- We then break the layout live and fix it using what you read.
- This is the #1 gap in the survey (2.2) - the reading is the prep.

Day 1 recap

- Flutter draws every pixel itself - no bridge, identical UI, AOT in release.
- flutter doctor clean, emulator or phone running, hot reload r / restart R.
- Dart = TypeScript-shaped with C# classes: final by default, null safety enforced, named params everywhere.
- Records, sealed classes and switch expressions model state exhaustively - Day 3 builds on this.
- Future = Promise/Task, Stream = Observable; isolates for CPU work only.
- Everything is a widget; three trees; Stateless vs Stateful; setState rebuilds one subtree.
- Quote app skeleton with feature-first structure is committed.

Tomorrow

- Widgets & layout - the constraint model
- Row / Column / Stack / Expanded, overflow errors and fixes
- Lists, forms and validation
- Material 3 theming → BrandTheme for white-label
- Lab: the real quote capture screen

PLAN B • When it breaks in class

If this happens	Do this instead - the class keeps moving
Twenty machines all downloading 2.8 GB of NDK at once	START THE BUILD BEFORE YOU TEACH. At 09:05, before any theory: flutter create quote_app then flutter build apk --debug. It pulls the NDK, SDK Platform and Gradle dist (~4 GB, 7-8 min on a good line). Teach Module 1 theory over the top of it. Never let this run during a lab.
Red X on Android license status unknown across the room	EXPECTED on current Android command-line tools (23.0+) - a known Flutter detection bug, not their machine. Say so BEFORE anyone runs doctor. The real test is a build: flutter build apk --debug. If that works, the toolchain is fine. Do not send them chasing licences.
flutter doctor will not go clean on a machine	That delegate does ALL of today in dartpad.dev (Labs 1.2 and 1.3 are browser-only by design). Pair them for Lab 1.4. Fix the install at lunch, not in front of the room.
Emulator will not boot (no hypervisor, CloudPC, corporate policy)	1. Physical Android phone + USB debugging. 2. flutter run -d chrome. Both work for every lab today and tomorrow.
First Gradle build crawls or fails downloading	USB stick: copy the pre-warmed .gradle and Pub cache, then flutter pub get --offline. Do this for the whole room at once rather than one at a time.
Venue internet is down entirely	USB stick has: SDK installers (both OS), pub + Gradle caches, the course repo as a zip, the five decks, and PDF copies of every doc used in Read & Break Down. Nothing today needs the network.
Projector / HDMI dies	Decks are on the USB stick; delegates open them locally and you narrate. Keep going.
You lose your thread	Every slide carries the full script in the notes. Use presenter view - second display shows notes, projector shows the slide.
A delegate knows a detail you do not	"Let us check the docs" → open docs.flutter.dev on the projector and read it together. Never bluff; this room will spot it and you lose them for the week.

APPENDIX

Day 1 reference

A • Command & path map (macOS / Windows) B • Lab 1.2 solution C • Lab 1.3 solution D • Lab 1.4 solution E • Troubleshooting F • Links

A • Paths, variables and shell differences

Item	macOS (zsh)	Windows (PowerShell)
Flutter SDK	~/development/flutter	C:\src\flutter
Add to PATH	~/ .zshrc: export PATH="\$HOME/development/flutter/bin:\$PATH"	Settings → Environment Variables → User Path → C:\src\flutter\bin
Android SDK	~/Library/Android/sdk	%LOCALAPPDATA%\Android\Sdk
adb	~/Library/Android/sdk/platform-tools/adb	%LOCALAPPDATA%\Android\Sdk\platform-tools\adb.exe
Pub cache	~/ .pub-cache	%LOCALAPPDATA%\Pub\Cache
Gradle cache	~/ .gradle	%USERPROFILE%\ .gradle
Env variable	\$HOME, \$VAR	\$env:USERPROFILE, \$env:VAR
Line continuation	backslash \	backtick `
Home shortcut	~	\$HOME or ~ (PowerShell 7)
Open file explorer	open .	explorer . / ii .
Edit hosts / env	nano ~/.zshrc	notepad \$PROFILE
JDK for Gradle	Android Studio bundled JBR	flutter config --jdk-dir "C:\Program Files\Android\Android Studio\jbr"

E • Troubleshooting: symptom → fix

Symptom	Cause	Fix
<code>flutter</code> not recognised	PATH not applied	Open a NEW terminal; verify Path entry; no spaces in SDK path
<code>cmdline-tools</code> component is missing	SDK tool not installed	SDK Manager → SDK Tools → Android SDK Command-line Tools (latest)
Android license status unknown AND <code>--licenses</code> says "no longer needed"	Known Flutter bug with <code>cmdline-tools</code> 23.0 (new android CLI replaced <code>sdksmanager</code>) - licences ARE accepted	Cosmetic. Verify: <code>ls <sdk>/licenses</code> has files, then prove with <code>flutter build apk --debug</code> . If the build works, carry on. flutter/flutter#191487
Android license status unknown on OLDER <code>cmdline-tools</code>	Licences genuinely not accepted	<code>flutter doctor --android-licenses</code> → y to all
Multiple <code>adb</code> binaries found	<code>platform-tools</code> installed twice (standalone + Android Studio)	Remove the duplicate, keep Android Studio's. macOS: <code>brew uninstall --cask android-platform-tools</code> . Windows: remove the standalone <code>platform-tools</code> folder from PATH
Emulator stuck on boot logo	No hardware acceleration	Win: Hypervisor Driver / WHPX + BIOS VT-x; CloudPC: use phone or Chrome
<code>adb</code> devices empty (Windows)	Missing USB driver / charge-only cable	Install OEM driver (Google USB Driver via SDK Manager); swap cable
Gradle: Unsupported class file major version	Wrong JDK	<code>flutter config --jdk-dir <Android Studio>/jbr</code>
First build takes 10+ min or fails downloading	It downloads ~4 GB: NDK 2.8 GB + SDK Platform + Gradle dist	Expected. Start it BEFORE the theory, not during a lab. Fallbacks: pre-warmed Android\Sdk, .gradle and Pub cache from USB; tether one machine; or pair two delegates on one warmed machine
<code>MissingPluginException</code>	Hot reload after adding native plugin	Stop and flutter run again
Hot reload "did nothing"	Change in <code>main()</code> /initialiser/const/enum	Press R (hot restart)
<code>setState()</code> called after <code>dispose()</code>	Async callback after navigation	<code>if (!mounted) return;</code> after every <code>await</code>
Filename too long (Windows git)	Long paths off	<code>git config --system core.longpaths true</code>
Build inexplicably broken	Stale artefacts	<code>flutter clean && flutter pub get</code>

B • Lab 1.2 solution - enum, request, quote

Lab 1.2 solution

```
enum Cover {
  thirdParty, thirdPartyFireTheft, comprehensive;
  double get factor => switch (this) {
    Cover.thirdParty => 0.6, Cover.thirdPartyFireTheft => 0.8, Cover.comprehensive => 1.0 };
}

class QuoteRequest {
  final String make; final String? model; final int year; final int driverAge; final Cover cover;
  const QuoteRequest({required this.make, this.model, required this.year,
    required this.driverAge, required this.cover});
  QuoteRequest copyWith({String? make, String? model, int? year, int? driverAge, Cover? cover}) =>
    QuoteRequest(make: make ?? this.make, model: model ?? this.model, year: year ?? this.year,
      driverAge: driverAge ?? this.driverAge, cover: cover ?? this.cover);
  @override
  bool operator ==(Object o) => o is QuoteRequest && o.make == make && o.model == model &&
    o.year == year && o.driverAge == driverAge && o.cover == cover;
  @override
  int get hashCode => Object.hash(make, model, year, driverAge, cover);
}

extension Money on double { String get rand => 'R ${toStringAsFixed(2)}'; }

class Quote {
  final String id; final double premium; final String currency;
  const Quote({required this.id, required this.premium, this.currency = 'ZAR'});
  String get display => '$currency ${premium.toStringAsFixed(2)}';
  factory Quote.fromJson(Map<String, dynamic> j) => Quote(id: j['id'] as String,
    premium: (j['premium'] as num).toDouble(), currency: j['currency'] as String? ?? 'ZAR');
  Map<String, dynamic> toJson() => {'id': id, 'premium': premium, 'currency': currency};
}
```

B • Lab 1.2 solution - calculator, sealed state, main (1/2)

```
double calculatePremium(QuoteRequest r) {  
  var p = 1000.0;  
  if (r.driverAge < 25) p *= 1.5;  
  if (r.year < 2015) p *= 1.2;  
  p *= r.cover.factor;  
  return (p * 100).roundToDouble() / 100;  
}
```

Lab 1.2 solution

```
sealed class QuoteState { const QuoteState(); }  
class QuoteIdle extends QuoteState { const QuoteIdle(); }  
class QuoteLoading extends QuoteState { const QuoteLoading(); }  
class QuoteLoaded extends QuoteState { const QuoteLoaded(this.quote); final Quote quote; }  
class QuoteFailed extends QuoteState { const QuoteFailed(this.message); final String message; }
```

B • Lab 1.2 solution - calculator, sealed state, main (2/2)

Lab 1.2 solution

```
String describe(QuoteState s) => switch (s) {
  QuoteIdle() => 'Fill in the form',
  QuoteLoading() => 'Calculating...',
  QuoteLoaded(:final quote) => 'Premium ${quote.display}',
  QuoteFailed(:final message) => 'Error: $message',
};

void main() {
  const base = QuoteRequest(make: 'VW', year: 2020, driverAge: 30, cover: Cover.comprehensive);
  final young = base.copyWith(driverAge: 22);
  final old = base.copyWith(year: 2012, cover: Cover.thirdParty);
  for (final r in [base, young, old]) print('${r.make} ${r.year} → ${calculatePremium(r).rand}');
  for (final s in [const QuoteIdle(), const QuoteLoading(),
    QuoteLoaded(Quote(id: 'q1', premium: calculatePremium(base))), const QuoteFailed('too old')]) {
    print(describe(s));
  }
}
```

C • Lab 1.3 solution - fake service, timeout, state stream (1/2)

```
abstract interface class QuoteService { Future<Quote> getQuote(QuoteRequest r); }
```

Lab 1.3 solution

```
class FakeQuoteService implements QuoteService {  
  @override  
  Future<Quote> getQuote(QuoteRequest r) async {  
    await Future.delayed(const Duration(milliseconds: 1500));  
    if (r.year < 2000) throw Exception('Vehicle too old to insure');  
    return Quote(id: 'q-${r.hashCode}', premium: calculatePremium(r));  
  }  
}
```

```
Future<void> runOnce(QuoteService s, QuoteRequest r) async {  
  try {  
    final q = await s.getQuote(r).timeout(const Duration(seconds: 3));  
    print('OK ${q.display}');  
  } on TimeoutException { print('Timed out'); }  
  catch (e) { print('Failed: $e'); }  
}
```


C • Lab 1.3 solution - fake service, timeout, state stream (2/2)

Lab 1.3 solution

```
Stream<QuoteState> quoteStates(QuoteService s, QuoteRequest r) async* {  
  yield const QuoteLoading();  
  try { yield QuoteLoaded(await s.getQuote(r)); }  
  catch (e) { yield QuoteFailed(e.toString()); }  
}  
  
void main() async {  
  final svc = FakeQuoteService();  
  const ok = QuoteRequest(make: 'VW', year: 2020, driverAge: 30, cover: Cover.comprehensive);  
  await for (final st in quoteStates(svc, ok)) print(describe(st));  
  await for (final st in quoteStates(svc, ok.copyWith(year: 1998))) print(describe(st));  
  final sw = Stopwatch()..start();  
  await Future.wait([svc.getQuote(ok), svc.getQuote(ok), svc.getQuote(ok)]);  
  print('3 parallel quotes in ${sw.elapsedMilliseconds} ms'); // ≈1500, not 4500  
}
```

D • Lab 1.4 solution - app.dart and capture_screen.dart (1/2)

```
// lib/app.dart
class QuoteApp extends StatelessWidget {
  const QuoteApp({super.key});
  @override
  Widget build(BuildContext context) => MaterialApp(
    title: 'Quote App',
    theme: ThemeData(colorSchemeSeed: const Color(0xFF0B2545), useMaterial3: true),
    home: const CaptureScreen());
}

// lib/features/quote/presentation/capture_screen.dart
class CaptureScreen extends StatefulWidget {
  const CaptureScreen({super.key});
  @override
  State<CaptureScreen> createState() => _CaptureScreenState();
}

class _CaptureScreenState extends State<CaptureScreen> {
  Cover _cover = Cover.comprehensive;
  double? _premium;

  void _calculate() {
    final r = QuoteRequest(make: 'VW', year: 2020, driverAge: 30, cover: _cover);
    setState(() => _premium = calculatePremium(r));
  }
}
```

Lab 1.4 solution

D • Lab 1.4 solution - app.dart and capture_screen.dart (2/2)

Lab 1.4 solution

```
@override
Widget build(BuildContext context) => Scaffold(
  appBar: AppBar(title: const Text('Get a quote')),
  body: Padding(padding: const EdgeInsets.all(16), child: Column(children: [
    SegmentedButton<Cover>(
      segments: [for (final c in Cover.values) ButtonSegment(value: c, label: Text(c.name))],
      selected: {_cover},
      onSelectionChanged: (s) => setState(() => _cover = s.first)),
    const SizedBox(height: 24),
    FilledButton(onPressed: _calculate, child: const Text('Calculate')),
    const SizedBox(height: 24),
    if (_premium != null) PremiumBadge(amount: _premium!),
  ])));
}

class PremiumBadge extends StatelessWidget {
  const PremiumBadge({super.key, required this.amount});
  final double amount;
  @override
  Widget build(BuildContext context) =>
    Text(amount.rands, style: Theme.of(context).textTheme.headlineMedium);
}
```

F • Links used today

Topic	Link
Install (macOS / Windows)	docs.flutter.dev/get-started/install/macos · docs.flutter.dev/get-started/install/windows
Emulator acceleration (Windows)	developer.android.com/studio/run/emulator-acceleration
Architecture overview	docs.flutter.dev/resources/architectural-overview
Dart language tour	dart.dev/language (null safety: dart.dev/null-safety ; patterns: dart.dev/language/patterns)
DartPad	dartpad.dev
Async / Streams	dart.dev/libraries/async/async-await · dart.dev/libraries/async/using-streams
Isolates	dart.dev/language/isolates
Widgets intro	docs.flutter.dev/ui/widgets-intro
Inside Flutter (three trees)	docs.flutter.dev/resources/inside-flutter
DevTools	docs.flutter.dev/tools/devtools
Packages	pub.dev · Flutter Favorites: docs.flutter.dev/packages-and-plugins/favorites
Course repo	github.com/Clynton19860/flutter
Reading for tomorrow	docs.flutter.dev/ui/layout/constraints